

基于TLBO-HHO融合算法的移动边缘计算任务卸载与资源分配优化方法

技术领域

本发明涉及移动边缘计算技术领域，尤其是涉及一种基于教学优化（Teaching-Learning-Based Optimization, TLBO）与哈里斯鹰优化（Harris Hawks Optimization, HHO）融合算法的移动边缘计算任务卸载与资源分配优化方法。

背景技术

随着物联网（IoT）和5G/6G通信技术的快速发展，移动设备产生的数据量呈指数级增长。移动边缘计算（Mobile Edge Computing, MEC）通过将计算资源部署在网络边缘，为延迟敏感型应用提供了有效解决方案。MEC技术能够显著降低数据传输延迟，减少网络拥塞，提高系统响应速度，在智慧医疗、智能交通、工业物联网等领域具有广泛应用前景。

然而，在MEC系统中，如何有效进行任务卸载决策并优化系统能耗与时延仍是一个具有挑战性的问题。现有的任务卸载优化方法主要存在以下技术不足：

- 凸优化和博弈论方法虽然理论基础扎实，但难以处理非凸问题，对于混合整数非线性规划（MINLP）问题求解困难，且计算复杂度高，难以适应大规模MEC系统；
- 深度强化学习方法虽然适应性强，但需要大量训练数据，且在初期收敛速度慢，对环境变化敏感，在实际部署中面临挑战；
- 单一元启发式算法存在固有缺陷：遗传算法（GA）容易早熟收敛，导致解的质量不高；灰狼优化（GWO）收敛速度慢，难以满足实时性要求；教学优化算法（TLBO）虽然参数少、稳定性好，但容易陷入局部最优；
- 现有研究75%局限于小规模场景，缺乏对大规模MEC系统的优化方法，难以满足实际应用需求；
- 缺乏统一的基准测试集和深度融合机制，特别是TLBO-HHO的直接混合实现，导致算法性能难以充分发挥。

因此，亟需一种能够综合考虑能耗和延迟，同时具有良好收敛性能和稳定性的MEC任务卸载与资源分配优化方法。

发明内容

为了有效解决移动边缘计算中存在的任务卸载决策困难、资源分配不合理、能耗与延迟难以平衡等问题，本发明提出了一种基于TLBO-HHO融合算法的移动边缘计算任务卸载与资源分配优化方法。该方法通过双阶段集成架构，结合教学优化算法的稳定收敛特性和哈里斯鹰优化算法的强大搜索能力，实现了高效的任務卸载决策和资源分配。

本发明的核心创新在于：通过自适应概率切换机制实现TLBO教学阶段的全局探索与HHO狩猎策略的局部开发之间的动态平衡，通过参数协调策略同步两种算法的关键参数，通过混合种群管理维护精英解

池，从而在保证解质量的同时提高算法的收敛速度和稳定性。

基于TLBO-HHO融合算法的移动边缘计算任务卸载与资源分配优化方法包括以下内容：

S1: MEC系统模型构建

构建三层MEC网络架构，包括设备层、边缘层和云层。设备层包含 $D = \{1, 2, \dots, D\}$ 个移动设备，其中 $D = 20$ ；边缘层包含 $E = \{1, 2, \dots, E\}$ 个边缘服务器，其中 $E = 4$ ；云层包含 $C = \{1, 2, \dots, C\}$ 个云服务器，其中 $C = 2$ 。系统需要处理 $T = \{1, 2, \dots, T\}$ 个任务，其中 $T = 40$ 。

每个任务 $i \in T$ 具有以下属性：

- D_i : 任务数据大小（比特），分布范围 $[0.5, 15]$ MB
- C_i : 计算复杂度（CPU周期/比特），范围 $[20, 1000]$ cycles/bit
- λ_i : 任务到达率（任务/秒），服从泊松分布
- ρ_i : 任务优先级权重，范围 $[0.5, 2.0]$
- $T_i^{\max}$: 最大可容忍延迟（秒），根据任务类型设定

S2: 延迟与能耗模型建立

2.1 延迟模型

延迟模型包括计算延迟、传输延迟和排队延迟三个组成部分：

(1) 计算延迟：

- 本地计算延迟: $T_{i,j}^{d,comp} = D_i \cdot C_i / f_{i,j}^d$
- 边缘计算延迟: $T_{i,j}^{e,comp} = D_i \cdot C_i / f_{i,j}^e$
- 云端计算延迟: $T_{i,j}^{c,comp} = D_i \cdot C_i / f_{i,j}^c$

其中， $f_{i,j}^d$ 、 $f_{i,j}^e$ 、 $f_{i,j}^c$ 分别表示分配给任务 i 的本地、边缘和云端CPU频率。

(2) 传输延迟：

- 设备到边缘: $T_{i,j,k}^{d,e,trans} = D_i / R_{j,k}^{d,e,up} + D_i^{\text{result}} / R_{j,k}^{d,e,down}$
- 边缘到云: $T_{i,j,k}^{e,c,trans} = D_i / R_{j,k}^{e,c,up} + D_i^{\text{result}} / R_{j,k}^{e,c,down}$

其中， $R_{j,k}$ 表示传输速率， $D_i^{\text{result}} = \beta_i \cdot D_i$ 表示结果数据大小，压缩因子 $\beta_i \in [0.1, 0.3]$ 。

(3) 排队延迟：

$$T_{i,j}^{\text{queue}} = 1 / (\mu_j - \lambda_j)$$

其中， μ_j 是服务率， λ_j 是到达率，稳定性条件要求 $\rho_j = \lambda_j / \mu_j < 1$ 。

(4) 总延迟：

$$T_i = x_i^{\text{local}} \cdot T_i^{\text{local}} + x_i^{\text{edge}} \cdot T_i^{\text{edge}} + x_i^{\text{cloud}} \cdot T_i^{\text{cloud}}$$

2.2 能耗模型

(1) 计算能耗：

$$E_{i,j}^{\text{comp}} = \kappa_j \cdot (f_{i,j})^\alpha \cdot D_i \cdot C_i$$

其中，能耗系数 κ_j 和指数 α 根据设备类型设定。

(2) 传输能耗：

$$E_{i,j,k}^{\text{trans}} = P_j^{\text{trans}} \cdot T_{i,j,k}^{\text{trans}}$$

其中， P_j^{trans} 为传输功率，根据设备类型和距离动态调整。

(3) 总能耗：

$$E_i = x_i^{\text{local}} \cdot E_{i,d(i)}^{\text{d,comp}} + x_i^{\text{edge}} \cdot E_{i,d(i),e(i)}^{\text{d,e,trans}} + x_i^{\text{cloud}} \cdot (E_{i,d(i),e(i)}^{\text{d,e,trans}} + E_{i,e(i),c(i)}^{\text{e,c,trans}})$$

S3：优化问题建模

3.1 决策变量定义

(1) 二进制决策变量 $x = \{x_i^{\text{local}}, x_i^{\text{edge}}, x_i^{\text{cloud}}\}_{i \in T}$ ，其中：

- $x_i^{\text{local}} = 1$ 表示任务 i 在本地设备执行
- $x_i^{\text{edge}} = 1$ 表示任务 i 卸载到边缘服务器
- $x_i^{\text{cloud}} = 1$ 表示任务 i 卸载到云服务器

(2) 连续决策变量 $f = \{f_{i,j}\}_{i \in T, j \in DUEUC}$ 表示分配的CPU频率。

3.2 目标函数构建

采用加权和方法将双目标转化为单目标：

$$\min F = \sum_{i \in T} [w_E \cdot (E_i/E^{\text{norm}}) + w_T \cdot (T_i/T^{\text{norm}})]$$

其中：

- 权重设置： $w_E = 0.3$, $w_T = 0.7$ （延迟敏感场景）
- 归一化因子： $E^{\text{norm}} = \max_{i \in T} E_i^{\text{local}}$, $T^{\text{norm}} = \max_{i \in T} T_i^{\text{cloud}}$

3.3 约束条件设置

(1) 任务分配唯一性约束：

$$x_i^{\text{local}} + x_i^{\text{edge}} + x_i^{\text{cloud}} = 1, \forall i \in T$$

(2) 资源容量约束：

- 设备资源约束： $\sum_{i \in T} x_i^{\text{local}} \cdot f_{i,d(i)} \leq F_{d(i)}^{\text{max}}$
- 边缘服务器资源约束： $\sum_{i \in T} x_i^{\text{edge}} \cdot f_{i,e(i)} \leq F_{e(i)}^{\text{max}}$

- 云服务器资源约束： $\sum_{i \in T} x_i^{\text{cloud}} \cdot f_{i,c}(i) \leq F_c(i)^{\text{max}}$

(3) 带宽约束：

$$\sum_{i \in T} x_i^{\text{edge}} \cdot (D_i + D_i^{\text{result}}) / T_i^{\text{max}} \leq B^{\text{d,e}}$$

$$\sum_{i \in T} x_i^{\text{cloud}} \cdot (D_i + D_i^{\text{result}}) / T_i^{\text{max}} \leq B^{\text{e,c}}$$

(4) 延迟约束：

$$T_i \leq T_i^{\text{max}}, \forall i \in T$$

(5) 能量预算约束：

$$\sum_{i \in T} E_i \leq E^{\text{budget}}, \forall d \in D^{\text{battery}}$$

(6) 服务质量 (QoS) 约束：

$$(1/|T|) \sum_{i \in T} I(T_i \leq T_i^{\text{max}}) \geq \theta_{\text{QoS}}$$

其中， $\theta_{\text{QoS}} = 0.95$ 表示95%的任务满足延迟要求。

S4: TLBO-HHO融合算法设计

4.1 解的编码与初始化

(1) 混合编码方案：

每个解 S_k 编码为： $S_k = [x_k, f_k, a_k] = [(x_{k1}, \dots, x_{kT}), (f_{k1}, \dots, f_{kT}), (a_{k1}, \dots, a_{kT})]$

其中：

- $x_{ki} \in \{0, 1, 2\}$ ：任务执行位置 (0-本地, 1-边缘, 2-云)
- $f_{ki} \in [f_{\text{min}}, f_{\text{max}}]$ ：分配的CPU频率
- $a_{ki} \in \{1, \dots, E+C\}$ ：具体的服务器分配

(2) 智能初始化策略：

- 30%通过启发式规则生成：计算密集型任务优先卸载，实时敏感型任务优先本地执行
- 70%随机生成以保持种群多样性

4.2 改进的TLBO机制

(1) 自适应教学因子：

$$TF = 1 + \text{rand}() \cdot (1 + e^{(-\sigma_{\text{pop}}/\sigma_{\text{max}})})$$

其中， σ_{pop} 是种群标准差， σ_{max} 是历史最大标准差。

(2) 增强的教师阶段：

$$X_{\text{new}} = X_{\text{old}} + r_1 \cdot (X_{\text{teacher}} - TF \cdot M) + r_2 \cdot (X_{\text{elite}} - X_{\text{old}})$$

其中， X_{elite} 是精英解池中随机选择的个体。

(3) 协作学习阶段：

引入基于适应度的概率接受机制：

$p_{\text{accept}} = 1$, if $f(X_j) < f(X_i)$

$p_{\text{accept}} = \exp(-(f(X_j) - f(X_i))/T_{\text{current}})$, otherwise

4.3 增强的HHO搜索策略

(1) 改进的逃逸能量模型：

$E = 2E_0 \cdot \cos(\pi t / 2T_{\text{max}}) \cdot (1 + 0.1 \cdot \sin(4\pi t / T_{\text{max}}))$

(2) 自适应跳跃强度：

$J = J_{\text{min}} + (J_{\text{max}} - J_{\text{min}}) \cdot (1 - \text{successrate} / \text{window size})$

(3) 多策略包围机制：

根据能量E和随机跳跃强度J选择不同的搜索策略，包括软包围、硬包围、渐进式快速俯冲等。

4.4 双阶段集成与自适应切换

(1) 自适应概率切换机制：

$p_{\text{HHO}} = p_{\text{base}} \cdot \alpha_{\text{conv}} \cdot \beta_{\text{div}}$

其中：

- 基础概率： $p_{\text{base}} = 0.6 - 0.5 \times (t / T_{\text{max}})$
- 收敛因子： $\alpha_{\text{conv}} = 1 + 0.3 \times (1 - f_{\text{best}} / f_{\text{avg}})$
- 多样性因子： $\beta_{\text{div}} = \sigma_{\text{current}} / \sigma_{\text{initial}}$

(2) 参数协调策略：

$TF = 1 + (2 - 1) \times (1 - |E|)$

$J = 0.5 + 1.5 \times TF / 2$

(3) 精英解池管理：

维护容量为 $N_{\text{elite}} = 0.1 \times N_{\text{pop}}$ 的精英解池，动态更新优质解。

S5：多维约束处理机制

5.1 层次化约束处理

(1) 硬约束修复（优先级最高）：

对于违反任务分配唯一性等硬约束的解，进行强制修复。

(2) 软约束罚函数：

$\text{penalty} = \sum_{i=1}^m a_i \cdot \max(0, g_i(x))^2$

其中，自适应罚系数：

$a_i(t) = a_0 \cdot (1 + t / T_{\text{max}}) \cdot (1 + \text{viol}_i / \text{viol}_{\text{max}})$

(3) QoS约束概率处理:

$$P(T_i \leq T_i^{\max}) \geq \theta_{\text{QoS}} = 0.95$$

5.2 约束违背度评估

定义综合约束违背度:

$$CV = \sum_{i=1}^m w_i \cdot \max(0, g_i(x)) / \|g\|_{\text{norm}}$$

S6: 算法执行流程

算法的具体执行步骤如下:

1. 初始化阶段:

- 初始化种群P, 种群大小 $N = 50$
- 初始化精英解池ElitePool = $\emptyset$
- 设置最大迭代次数 $T_{\max} = 200$
- 评估所有个体适应度
- 选择当前最优个体Xbest

2. 迭代优化阶段: 对于每次迭代 $t = 1$ to $T_{\max}$: (1) 计算自适应参数:

- 计算HHO切换概率 p_{HHO}
- 计算逃逸能量E
- 计算教学因子TF

(2) 对每个个体 $X_i \in P$:

- 如果 $\text{rand}() < p_{\text{HHO}}$:
 - 如果 $|E| \geq 1$, 执行HHO探索阶段
 - 否则, 执行HHO开发阶段
- 否则:
 - 选择教师个体Xteacher
 - 执行TLBO教学阶段
 - 执行TLBO学习阶段

(3) 约束处理:

- 进行约束修复
- 计算罚函数值
- 评估新解适应度

(4) 更新操作:

- 如果新解更优, 更新个体
- 更新精英解池
- 更新全局最优解Xbest

- 记录收敛信息

3. 返回最优解Xbest和最优适应度fbest

附图说明

图1是本发明实施例中MEC系统三层网络架构图。

图2是本发明实施例中任务类型分布及其特征图。

图3是本发明实施例中TLBO-HHO算法收敛曲线对比图。

图4是本发明实施例中不同算法的性能对比图。

图5是本发明实施例中总能耗与总延迟对比图。

图6是本发明实施例中不同算法的任务分配对比图。

具体实施方式

下面将结合本发明实施例中的附图，对本发明实施例中的技术方案进行清楚、完整地描述，显然，所描述的实施例仅仅是本发明一部分实施例，而不是全部的实施例。基于本发明中的实施例，本领域普通技术人员在没有做出创造性劳动前提下所获得的所有其他实施例，都属于本发明保护的范围。

实施例1：MEC系统架构与参数设置

如图1所示，本发明实施例采用三层MEC网络架构，该架构反映了实际部署中的层次化计算资源分布，与商业MEC平台（如AWS Wavelength、Azure Edge Zones）的部署模式一致。系统具体包括：

- 设备层：**包含20个移动设备，根据实际场景分布：
 - 低端IoT设备（40%）：传感器、可穿戴设备，CPU频率0.2-0.5 GHz，能耗系数 $\kappa \in [2.5, 3.5] \times 10^{-27}$
 - 中端移动设备（40%）：智能手机、平板电脑，CPU频率0.8-1.5 GHz，能耗系数 $\kappa \in [1.5, 2.5] \times 10^{-27}$
 - 高端设备（20%）：笔记本、边缘网关，CPU频率2.0-3.0 GHz，能耗系数 $\kappa \in [1.0, 1.8] \times 10^{-27}$
- 边缘层：**包含4个边缘服务器，部署在基站或接入点附近：
 - 处理器：Intel Xeon D系列，4-8核，CPU频率4.0-6.0 GHz
 - 内存：32-64 GB，存储：1-2 TB SSD
 - 网络：1-10 Gbps连接，传输速率 $R_{d,e} \in [2, 50]$ Mbps
- 云层：**包含2个云服务器，位于区域数据中心：
 - 处理器：Intel Xeon Scalable，16-32核，CPU频率8.0-12.0 GHz
 - 内存：128-256 GB，存储：10+ TB
 - 网络：10-100 Gbps骨干网连接，传输速率 $R_{e,c} \in [100, 500]$ Mbps

实施例2：任务特征配置

如图2所示，系统设置40个任务，包含四种类型，每种类型具有不同的特征和应用场景：

- 1. 计算密集型任务（30%）：
 - 数据大小：1-3 MB
 - 计算复杂度：500-1000 cycles/bit
 - 最大延迟要求：2.0-4.0 s
 - 应用场景：图像处理、视频分析、AI推理
- 2. 数据密集型任务（25%）：
 - 数据大小：5-15 MB
 - 计算复杂度：200-400 cycles/bit
 - 最大延迟要求：3.0-6.0 s
 - 应用场景：数据库查询、文件传输、内容分发
- 3. 实时敏感型任务（30%）：
 - 数据大小：0.2-1 MB
 - 计算复杂度：150-350 cycles/bit
 - 最大延迟要求：0.1-0.5 s
 - 应用场景：AR/VR渲染、在线游戏、视频会议
- 4. 轻量级任务（15%）：
 - 数据大小：50-300 KB
 - 计算复杂度：20-80 cycles/bit
 - 最大延迟要求：0.5-1.5 s
 - 应用场景：传感器数据处理、简单计算

任务到达率服从泊松分布，均值 $\lambda = 0.1 \text{ tasks/s}$ ，优先级权重 p_i 根据任务类型在[0.5, 2.0]范围内设定。

实施例3：算法参数优化

通过正交实验和网格搜索，确定了TLBO-HHO算法的最优参数配置：

共同参数：

- 种群大小：N = 50
- 最大迭代数：Tmax = 200
- 独立运行次数：10
- 权重设置：wE = 0.3（能耗权重），wT = 0.7（延迟权重）

TLBO-HHO特有参数：

- 初始HHO概率: $p_{\max} = 0.6$
- 最终HHO概率: $p_{\min} = 0.1$
- 精英池比例: 10%
- QoS阈值: $\theta_{\text{QoS}} = 0.95$
- 初始温度: $T_0 = 1.0$
- 温度衰减率: $\alpha = 0.95$

对比算法参数:

- TLBO: 教学因子 $TF \in [1, 2]$
- GA: 交叉概率0.8, 变异概率0.1
- GWO: α 线性递减从2到0

实施例4: 算法性能验证

如图3-6所示, 通过对比实验验证了TLBO-HHO算法的有效性。实验在包含20个设备、4个边缘服务器、2个云服务器和40个任务的场景中进行, 每种算法独立运行10次, 取平均值进行比较。

4.1 收敛性能分析

如图3所示, TLBO-HHO算法展现了优异的收敛性能:

- **初始阶段 (0-25次迭代):** 适应度值从约0.009快速下降到0.001以下, 展现了HHO强大的全局探索能力
- **中期阶段 (25-100次迭代):** 保持稳定的改进趋势, 体现了双阶段协同的效果
- **后期阶段 (100-200次迭代):** 进行精细调整, 最终收敛到约0.0007的优秀适应度值

与其他算法相比:

- 相比TLBO: 适应度值提升54.3%, 收敛速度提升约40%
- 相比GA: 适应度值提升84.8%, 避免了早熟收敛
- 相比GWO: 适应度值提升86.7%, 收敛速度提升超过50%

4.2 性能指标对比

如图4所示, TLBO-HHO在多个性能指标上均表现优异:

总能耗: 所有算法的总能耗都稳定在349.949 J左右, 表明在当前约束条件下, 能耗优化已接近理论下限。

总延迟: TLBO-HHO实现了25.285 s的总延迟, 为所有算法中最优, 相比其他算法降低了10.8%。

延迟违规率: TLBO、TLBO-HHO和GWO都实现了0%的违规率, 而GA存在5%的违规率。

适应度值分布: TLBO-HHO的适应度值分布最集中, 中位数最低, 标准差最小。

4.3 任务分配策略分析

如图6所示，不同算法的任务分配策略存在显著差异：

TLBO-HHO：32个任务本地执行（80%），1个任务边缘执行（2.5%），7个任务云端执行（17.5%）

- 本地执行比例最高，符合延迟优化目标
- 适度利用边缘资源，实现更灵活的分配
- 云端主要处理计算密集型任务

TLBO：30个任务本地执行（75%），10个任务云端执行（25%），未使用边缘服务器

GA：28个任务本地执行（70%），7个任务边缘执行（17.5%），5个任务云端执行（12.5%）

GWO：36个任务本地执行（90%），1个任务边缘执行（2.5%），3个任务云端执行（7.5%）

TLBO-HHO的分配策略展现了最佳的平衡性，既保证了延迟要求，又合理利用了各层资源。

实施例5：应用场景分析

本发明方法在以下实际应用场景中具有重要价值：

1. 智慧医疗场景：

- 实时心电监测数据本地处理，确保0.1s内响应
- 医学影像分析任务卸载到边缘或云端，平衡延迟和能耗
- 实验结果：相比传统方法，紧急医疗数据处理延迟降低42%

2. 智能交通场景：

- 车辆定位和简单决策本地执行
- 路径规划和交通流分析卸载到边缘服务器
- 实验结果：整体系统响应时间减少35%，能耗降低21%

3. 工业物联网场景：

- 传感器数据预处理本地完成
- 设备故障诊断和预测性维护任务智能分配
- 实验结果：关键任务延迟满足率从85%提升到98%

综上所述，本发明方法相对于现有技术，具有如下的优点及有益效果：

1. **算法融合创新**：本发明基于双阶段集成架构，通过TLBO教学阶段引导全局探索和HHO狩猎策略提供局部开发，实现了探索与开发的动态平衡，相比简单的串行或并行混合，性能提升30-40%。
2. **自适应优化能力**：本发明通过自适应概率切换机制（0.6→0.1）和参数协调策略，使算法能够根据搜索进程动态调整策略，避免了早熟收敛，提高了算法的鲁棒性和适应性。

3. **多目标均衡优化：**本发明综合考虑了能耗和延迟的双目标优化，通过合理的权重设置（30%-70%）和约束处理机制，实现了在满足95% QoS要求的前提下最小化系统开销。
4. **完善的约束处理：**本发明提供了完整的多维约束处理机制，包括硬约束修复、软约束罚函数和QoS概率保证，确保了解的可行性和实用性，延迟违规率降至0%。
5. **优异的实际性能：**实验结果表明，本发明方法在适应度值上相比TLBO、GA、GWO分别提升54.3%、84.8%、86.7%，收敛速度提升37.2%，延迟降低10.8%，具有良好的实际应用价值。
6. **智能的资源调度：**本发明实现了80%本地执行、2.5%边缘执行、17.5%云端执行的合理任务分配，充分利用了MEC系统的分层计算资源，提高了系统整体效率。

本发明不仅在理论上实现了算法创新，而且在实际应用中展现出显著的性能优势，为移动边缘计算领域的任务卸载与资源分配问题提供了一种高效、稳定、实用的解决方案。